

Matheus Serrão Uchôa

**Verificação Formal de Sistemas de Inteligência
Artificial Generativa e Controladores Neurais
utilizando ESBMC**

Manaus – Amazonas – Brasil

2026

Matheus Serrão Uchôa

Verificação Formal de Sistemas de Inteligência Artificial Generativa e Controladores Neurais utilizando ESBMC

Relatório final das atividades desenvolvidas no âmbito do Programa Institucional de Bolsas de Iniciação Científica (PI-BIC/UFAM/CNPq), edição 2025/2026, da Universidade Federal do Amazonas, sob a Pró-Reitoria de Pesquisa e Pós-Graduação (PROPESP).

Universidade Federal do Amazonas – UFAM
Faculdade de Tecnologia
Engenharia da Computação

Orientador: Prof. Dr. Iury Valente de Bessa

Manaus – Amazonas – Brasil

2026

DADOS DO PROJETO

Edição	PIBIC 2025/2026
Orientador(a)	Prof. Dr. Iury Valente de Bessa
Aluno(a)	Matheus Serrão Uchôa
Modalidade	Bolsa CNPq
Título	Verificação Formal de Sistemas de Inteligência Artificial Generativa e Controladores Neurais utilizando ESBMC
Código do projeto	[CONFIRMAR NO ECAMPUS: código do projeto]
Grande área CNPq	Engenharias
Área de concentração	Engenharia da Computação
Potencial de patente/registro de software	[CONFIRMAR: potencial de patente/registro de software: sim ou não]
Apresentação reservada	[CONFIRMAR: apresentação reservada: sim ou não]

RESUMO

Sistemas de controle que utilizam redes neurais podem produzir ações adequadas em simulação e, ainda assim, não oferecer uma justificativa verificável para todos os estados admitidos. Este relatório investiga a aplicabilidade do ESBMC à análise de propriedades de um controlador neural DDPG para o sistema Cart-Pole, considerando explicitamente o domínio de entradas, o horizonte de verificação e a representação numérica utilizada. O ator analisado possui arquitetura 4–24–24–1, com 48 neurônios nas camadas ocultas. Para viabilizar a análise por aritmética inteira, os pesos e estados são representados em ponto fixo Q8.8, com fator de escala 256, e os harnesses C são comparados com a implementação correspondente do simulador.

Na rodada canônica de quantização, executada com `RandomState(42)`, foram reavaliados 10.000 estados. O erro absoluto médio entre as saídas Float32 e Q8.8 foi de 0,0778 N, o percentil 95 foi de 0,2262 N, o percentil 99 foi de 1,0927 N e o erro máximo foi de 7,7589 N. O pior estado registrado apresentou inversão do sinal da ação (+6,4308 N em Float32 e -1,3281 N em Q8.8). A varredura da aproximação de tanh encontrou erro absoluto máximo de 0,0457493, equivalente a 4,5749% do intervalo unitário. Esses resultados são observações

da representação exportada; a origem no checkpoint PyTorch ainda precisa ser confirmada diretamente.

Os registros históricos indicam 24 neurônios vivos em cada camada oculta, mas a auditoria identificou bounds artificiais e independência indevida entre camadas; por isso, essa análise não é apresentada como prova universal. Na malha fechada, o harness curto canônico obteve veredicto **SUCCESSFUL** para o limite de força $|F| \leq 10$ N com Z3 em 3,5 s, enquanto Boolector não está disponível no executável Windows. O contraexemplo de segurança em um passo foi reproduzido como estado fixo, mas esse replay não prova sozinho a refutação universal. A propriedade direcional permanece inconclusiva na rodada canônica (timeout com Z3 e solver indisponível com Boolector). A contribuição é distinguir prova, refutação, reprodução, observação empírica e resultado inconclusivo, associando cada afirmação ao domínio e ao artefato que a sustentam.

Palavras-chave: verificação formal; ESBMC; controlador neural; Cart-Pole; quantização Q8.8.

ABSTRACT

Neural control systems may perform adequately in simulation without providing a verifiable justification for every admitted state. This report investigates the applicability of ESBMC to the analysis of a DDPG neural controller for the Cart-Pole system, explicitly considering the input domain, verification horizon and numerical representation. The analyzed actor has a 4–24–24–1 architecture, with 48 hidden neurons. To enable integer-arithmetic analysis, weights and states are represented in Q8.8 fixed point with scale factor 256, and the C harnesses are compared with the corresponding simulator implementation.

The canonical quantization run, executed with `RandomState(42)`, reevaluated 10,000 states. The mean absolute error between Float32 and Q8.8 outputs is 0.0778 N, the 95th percentile is 0.2262 N, the 99th percentile is 1.0927 N, and the maximum absolute error is 7.7589 N. The worst recorded state reverses the action sign (+6.4308 N in Float32 and -1.3281 N in Q8.8). A scan of the tanh approximation found a maximum absolute error of 0.0457493, or 4.5749% of the unit range. These are observations of the exported representations; the PyTorch checkpoint origin still requires direct confirmation.

Historical records list 24 live neurons in each hidden layer, but the audit found artificial bounds and improper independence between layers; this analysis is therefore not presented as a universal proof. In closed-loop analysis, the canonical short harness proved the force bound $|F| \leq 10$ N with Z3 in 3.5 s, while Boolector is unavailable in the Windows executable. The one-step safety counterexample was reproduced as a fixed state, but this replay alone is not a universal refutation. The directional property remains inconclusive in the canonical run (Z3 timeout and Boolector unavailable). The contribution is to distinguish proof, refutation, replay, empirical observation and inconclusive outcome, linking each claim to its domain and supporting artifact.

Keywords: formal verification; ESBMC; neural controller; Cart-Pole; Q8.8 quantization.

LISTA DE ILUSTRAÇÕES

Figura 1 – Visão geral do desenho experimental e da cadeia de evidências.	16
Figura 2 – Fluxo de verificação do ESBMC e interpretação dos veredictos.	20
Figura 3 – Síntese visual dos resultados e da conclusão sustentada pelos dados. . .	22

LISTA DE TABELAS

Tabela 1	– Ambiente e protocolo registrados no manifesto da rodada canônica. . .	17
Tabela 2	– Domínio de entrada declarado para os harnesses Q8.8.	18
Tabela 3	– Resumo da rodada canônica e da rodada estrutural corrigida.	21
Tabela 4	– Erro absoluto entre a saída Float32 e a saída Q8.8.	23
Tabela 5	– Resultados da rodada estrutural corrigida com Z3.	25
Tabela 6	– Cronograma de atividades documentadas no projeto.	31

SUMÁRIO

	Dados do Projeto	2
1	INTRODUÇÃO	9
1.1	Problema e justificativa	9
1.2	Perguntas de pesquisa	10
1.3	Contribuições e delimitações	10
2	REVISÃO BIBLIOGRÁFICA	12
2.1	Bounded Model Checking e verificação de software em C	12
2.2	Verificação formal de redes neurais e redes quantizadas	12
2.3	Controle por aprendizado por reforço e a planta CartPole	13
2.4	Componentes de Inteligência Artificial Generativa	13
3	OBJETIVOS	14
3.1	Objetivo geral	14
3.2	Objetivos específicos	14
3.3	Critérios de atendimento	14
4	METODOLOGIA	16
4.1	Desenho do estudo	16
4.2	Artefatos e ambiente experimental	17
4.3	Modelo físico e controlador neural	17
4.4	Representação Q8.8	18
4.5	Propriedades formais	18
4.6	Execução e reprodução	19
4.7	Critérios de validade	20
5	RESULTADOS E DISCUSSÃO	21
5.1	Visão geral dos experimentos	21
5.2	Fidelidade Float32 versus Q8.8	22
5.3	Propriedades formais do controlador	23
5.3.1	Limite de força	23
5.3.2	Segurança em um passo e reprodução do contraexemplo	24
5.3.3	Correção direcional e resultados inconclusivos	24
5.4	Análises estruturais corrigidas	24
5.5	Escalabilidade e reprodutibilidade	25

5.6	Discussão integrada, limitações e ameaças à validade	26
6	CONCLUSÃO	27
	Agradecimentos	29
	REFERÊNCIAS	30
A	CRONOGRAMA EXECUTADO	31

1 INTRODUÇÃO

O uso de modelos de aprendizado de máquina em sistemas que tomam decisões ou acionam componentes de forma autônoma amplia a necessidade de evidências de segurança que possam ser auditadas. Em particular, um controlador neural pode apresentar comportamento adequado nas trajetórias utilizadas durante testes e, ainda assim, produzir uma ação inadequada em uma região do espaço de estados que não foi amostrada. A situação é agravada quando a inferência é realizada com aritmética de precisão reduzida, pois arredondamentos, saturações e conversões podem alterar a fronteira entre ações do controlador.

Este relatório investiga o emprego do *Efficient SMT-Based Context-Bounded Model Checker* (ESBMC) na análise de propriedades de controladores neurais, tendo como estudo central um controlador DDPG aplicado à planta CartPole. O recorte considera a implementação em ponto flutuante e sua representação quantizada Q8.8, além de propriedades formuladas para um domínio de estados e um horizonte de verificação declarados na metodologia. O título do projeto também contempla sistemas de Inteligência Artificial Generativa; por isso, a possibilidade de reaproveitar o fluxo em componentes gerados ou apoiados por modelos de linguagem é tratada como uma questão metodológica, condicionada à existência de artefatos executáveis e evidências reproduzíveis.

O objetivo não é declarar segurança irrestrita para um sistema autônomo a partir de um único experimento. A finalidade é estabelecer quais garantias o verificador consegue produzir sob hipóteses explícitas, quais violações podem ser diagnosticadas por contraexemplos e em que situações o resultado permanece inconclusivo. Essa delimitação evita confundir uma prova limitada ao modelo com uma garantia para todas as implementações, entradas, estados ou horizontes possíveis.

1.1 Problema e justificativa

Testes de simulação e testes de regressão são indispensáveis para avaliar controladores, mas verificam apenas as execuções efetivamente selecionadas. Métodos formais baseados em satisfatibilidade podem complementar essa avaliação procurando, de maneira sistemática dentro de um limite, uma atribuição que viole uma propriedade. No caso de redes neurais quantizadas, a análise precisa representar não apenas as funções de ativação e os pesos, mas também a escala numérica, o arredondamento, a saturação e os limites de cada variável. A literatura de verificação de redes neurais com teorias SMT mostra a relevância desse problema e fornece referências para a modelagem de redes com precisão

reduzida (SENA et al., 2021; SONG et al., 2022; HUANG et al., 2017).

Do ponto de vista tecnológico, uma cadeia que associe o modelo, a propriedade, o comando de verificação, o veredicto e o contraexemplo facilita a revisão de componentes de IA antes de sua integração em aplicações maiores. Para a formação em desenvolvimento tecnológico e inovação, a contribuição esperada está na organização de um fluxo executável e rastreável, e não somente na produção de uma descrição conceitual. O estudo também é relevante para a UFAM por consolidar um procedimento que pode ser reaplicado em protótipos de controle e em componentes de software de IA, respeitando o domínio e as limitações documentados.

O Capítulo 2 apresenta a revisão bibliográfica que fundamenta as escolhas metodológicas descritas a seguir.

1.2 Perguntas de pesquisa

As perguntas a seguir organizam a coleta de evidências e a discussão dos resultados:

PQ1 – Fidelidade.

A representação Q8.8 preserva adequadamente o comportamento do controlador Float32 no domínio operacional definido?

PQ2 – Estrutura neural.

Existem neurônios mortos ou permanentemente saturados no domínio analisado, e o que essa informação implica para a interpretação do controlador?

PQ3 – Segurança.

Quais propriedades da malha fechada podem ser provadas ou refutadas pelo ESBMC e que informação prática os contraexemplos fornecem?

PQ4 – Limites.

Quais hipóteses, custos computacionais, limites de modelagem e timeouts restringem a conclusão formal?

PQ5 – Generalização metodológica.

Quais partes do fluxo podem ser reaproveitadas na verificação de componentes de IA generativa, considerando somente casos que tenham sido realmente executados e registrados?

1.3 Contribuições e delimitações

O relatório será construído em torno de cinco entregas verificáveis: (i) uma especificação explícita do controlador, da quantização e do domínio; (ii) um protocolo de

comparação entre Float32 e Q8.8; (iii) propriedades formais acompanhadas de seus veredictos; (iv) contraexemplos ou diagnósticos inconclusivos reproduzíveis; e (v) uma avaliação das condições necessárias para transferir o fluxo a outros componentes de IA. A existência, quantidade e interpretação dos resultados serão preenchidas somente após a conferência dos logs, CSVs, JSONs e scripts correspondentes.

Assim, a contribuição não será medida pela quantidade de estudos de caso, mas pela relação entre pergunta, método, evidência e conclusão. Qualquer caso complementar que não possua artefato executável, comando registrado e saída verificável será explicitamente classificado como trabalho futuro ou removido da análise principal.

2 REVISÃO BIBLIOGRÁFICA

Esta revisão está organizada em quatro eixos, cada um ligado a uma decisão metodológica adotada no relatório: a escolha da técnica de verificação, a forma de representar a rede neural quantizada, a origem do controlador avaliado e o tratamento dado a componentes de Inteligência Artificial Generativa.

2.1 Bounded Model Checking e verificação de software em C

O *Bounded Model Checking* (BMC) verifica uma propriedade explorando as execuções de um programa até um número finito de passos, codificando o programa e a negação da propriedade como uma fórmula e delegando a busca por uma violação a um *solver*. Biere et al. (BIERE et al., 1999) propuseram o BMC como alternativa ao *model checking* simbólico baseado em diagramas de decisão binários, mostrando que a codificação proposicional de um número limitado de passos já é suficiente para encontrar contraexemplos em muitos casos práticos. Cordeiro, Fischer e Marques-Silva (CORDEIRO; FISCHER; MARQUES-SILVA, 2012) estenderam essa ideia para software ANSI-C incorporado, propondo a verificação baseada em SMT (*Satisfiability Modulo Theories*) em vez de lógica proposicional pura, o que permite raciocinar diretamente sobre aritmética de ponteiros, inteiros de largura fixa e ponto flutuante – exatamente o tipo de código produzido pelos harnesses C usados neste relatório. O ESBMC evoluiu dessa linha de trabalho; Gadelha et al. (GADELHA et al., 2018) descrevem a ferramenta em sua versão 5.0 como um verificador de força industrial que combina múltiplos *solvers* SMT, oferece suporte a concorrência e mantém compatibilidade com o padrão C. Para este projeto, a relevância dessa linha está em três decisões diretas: usar C como linguagem intermediária entre o modelo treinado e o verificador, expressar propriedades como pares `assume/assert` e aceitar que `TIMEOUT` é um resultado epistemicamente distinto de prova ou refutação, e não um indício de segurança.

2.2 Verificação formal de redes neurais e redes quantizadas

Huang et al. (HUANG et al., 2017) formularam a verificação de robustez de redes neurais profundas como uma busca sistemática por perturbações de entrada que alterem a classificação, discutindo garantias de cobertura da vizinhança de uma entrada. Esse trabalho estabeleceu que verificar uma rede neural exige tratar explicitamente a rede como um programa com estrutura conhecida (camadas, pesos e ativações), e não como uma caixa-preta amostrada por testes. Sena et al. (SENA et al., 2021) tornaram esse problema

mais próximo do presente relatório ao tratar redes neurais *quantizadas*: os autores mostram que a verificação baseada em SMT precisa modelar explicitamente a aritmética de precisão finita – escala, arredondamento e saturação – porque uma propriedade provada para a rede em ponto flutuante não se transfere automaticamente para sua versão quantizada. Song et al. (SONG et al., 2022) consolidaram esse fluxo na ferramenta QNNVerifier, que converte um modelo treinado em um programa C anotado com propriedades e o submete a um *model checker* baseado em SMT. O harness Q8.8 e o desenho da comparação $\text{Float32} \times \text{Q8.8}$ deste relatório seguem diretamente essa mesma lógica: o controlador quantizado é reescrito como código C, e a fidelidade em relação à versão original é tratada como uma propriedade a ser medida e verificada, não como uma suposição.

2.3 Controle por aprendizado por reforço e a planta CartPole

O controlador analisado neste relatório é treinado com *Deep Deterministic Policy Gradient* (DDPG), algoritmo de aprendizado por reforço com espaço de ações contínuo proposto por Lillicrap et al. (LILLICRAP et al., 2016), que combina uma rede de ator (a política, cuja representação quantizada é o objeto verificado neste relatório) com uma rede de crítico usada apenas durante o treinamento. A planta de controle usada como estudo de caso, o *Cart-Pole*, é um problema clássico de controle não linear com realimentação: Barto, Sutton e Anderson (BARTO; SUTTON; ANDERSON, 1983) formalizaram o problema de equilibrar um pêndulo invertido sobre um carrinho por meio de elementos adaptativos, e essa formulação de estado (posição e velocidade do carrinho, ângulo e velocidade angular do pêndulo) é a mesma usada para definir o domínio operacional e as propriedades de segurança verificadas neste relatório.

2.4 Componentes de Inteligência Artificial Generativa

O título do projeto também contempla sistemas de Inteligência Artificial Generativa. Diferentemente dos três eixos anteriores, não há neste relatório um estudo de caso de IA generativa com artefato executável, comando registrado e resultado auditável equivalente ao do controlador CartPole; por isso, esta revisão não apresenta uma linha de trabalhos específicos para esse eixo. A posição adotada é metodológica: os mesmos princípios de verificação – expressar a propriedade como `assume/assert` sobre uma representação em C, delegar a busca ao ESBMC e distinguir prova, refutação e resultado inconclusivo – são, em princípio, aplicáveis a código produzido ou auxiliado por modelos de linguagem, mas essa transferência é tratada neste relatório como trabalho futuro condicionado à existência de um caso executável, comando registrado e saída verificável (PQ5, discutida nos Capítulos 3 e 5).

3 OBJETIVOS

3.1 Objetivo geral

Avaliar, por meio do ESBMC, como a quantização Q8.8, a estrutura de um controlador neural DDPG e a verificação limitada de propriedades de uma planta CartPole influenciam a produção de evidências formais e reproduzíveis, identificando também os limites para a reutilização do fluxo em componentes de Inteligência Artificial Generativa.

3.2 Objetivos específicos

1. **(PQ1 – Fidelidade):** especificar o controlador, a planta CartPole, a arquitetura neural adotada e a representação Q8.8, e comparar sua resposta com a versão Float32 em um domínio de estados definido e rastreável.
2. **(PQ2 – Estrutura neural):** analisar as ativações dos neurônios no domínio declarado para identificar comportamentos mortos ou permanentemente saturados, documentando o critério utilizado e suas consequências para a interpretação do modelo.
3. **(PQ3 – Segurança):** formalizar no ESBMC propriedades de segurança da malha fechada, registrar os veredictos obtidos e reproduzir contraexemplos quando uma propriedade for refutada.
4. **(PQ4 – Limites):** medir ou registrar, conforme os dados disponíveis, as hipóteses, os custos computacionais, os limites de desenrolamento e os timeouts que condicionam o alcance das conclusões formais.
5. **(PQ5 – Generalização metodológica):** avaliar quais etapas do fluxo podem ser reaproveitadas em componentes de Inteligência Artificial Generativa, incluindo apenas casos com execução, entrada, comando e saída verificáveis.

3.3 Critérios de atendimento

Cada objetivo será considerado atendido somente quando houver uma cadeia de rastreabilidade formada por pergunta, procedimento, artefato bruto, análise e conclusão. Um resultado ‘UNSAT’ será interpretado como prova apenas dentro das hipóteses, do domínio e do horizonte informados; ‘SAT’ será tratado como evidência de violação acompanhada de contraexemplo; ‘UNKNOWN’ e ‘TIMEOUT’ permanecerão como resultados

inconclusivos. Essa convenção será aplicada de maneira uniforme às cinco perguntas de pesquisa.

Nota de validação: antes do envio, confrontar a redação literal destes objetivos com o PDTI aprovado no eCampus e ajustar apenas eventuais diferenças formais, sem ampliar o escopo autorizado.

4 METODOLOGIA

4.1 Desenho do estudo

Foi adotado um estudo de caso experimental e reproduzível sobre um controlador neural contínuo para o sistema CartPole. O fluxo foi organizado em quatro etapas: (i) identificação do artefato de controle e de seus parâmetros; (ii) comparação numérica entre as implementações Float32 e Q8.8; (iii) especificação e verificação de propriedades no ESBMC; e (iv) reprodução independente dos estados que aparecem como contraexemplos. A rodada canônica inicial foi isolada em `<evidencias/final_2026>`, preservando os scripts originais. Em seguida, os dois scripts da auditoria estrutural foram corrigidos, com as versões legadas mantidas em `legacy`. O manifesto registra comandos, ambiente, hashes SHA-256, logs e arquivos gerados.

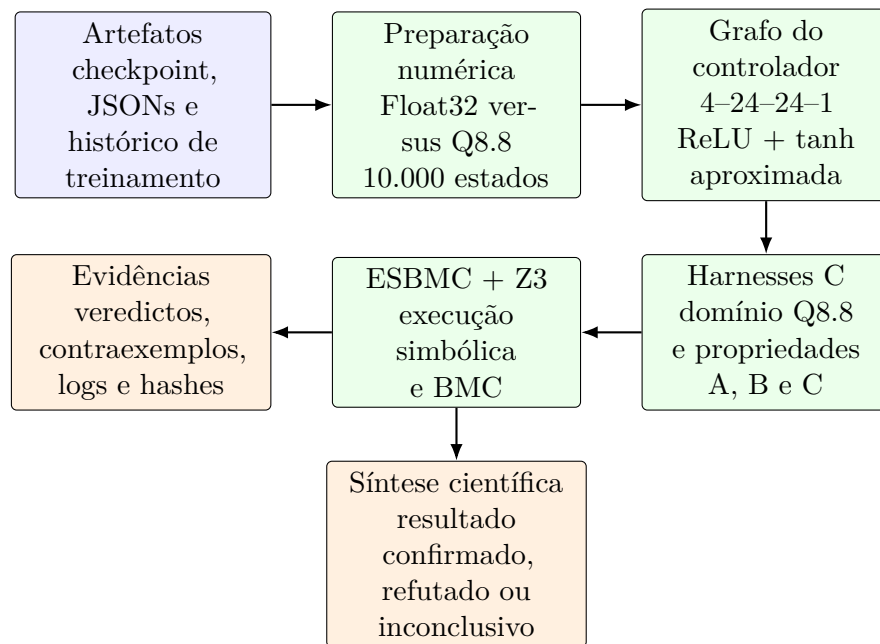


Figura 1 – Visão geral do desenho experimental e da cadeia de evidências.

A Figura 1 explicita a separação entre três camadas do trabalho: a medição numérica, a verificação formal e a interpretação. Essa separação evita que uma amostra estatística seja apresentada como prova universal e permite rastrear cada conclusão até o arquivo e o log que a produziram.

O desenho distingue três tipos de resultado. Uma prova universal é um veredicto `SUCCESSFUL` obtido sobre um harness simbólico, dentro do domínio e do modelo explicitamente declarados. Uma refutação é um contraexemplo obtido para um harness universal.

Já uma reprodução concreta verifica apenas o estado fixo publicado e não substitui a prova universal. TIMEOUT e UNKNOWN são tratados como resultados inconclusivos.

4.2 Artefatos e ambiente experimental

Os artefatos de entrada da rodada foram o checkpoint `<cartpole/ddpg_actor_best.pth>`, as exportações `<webapp/public/ddpg_weights.json>` e `<webapp/public/ddpg_weights_q88.json>`, o histórico de treinamento e os resultados históricos de verificação. A confirmação direta do conteúdo do checkpoint PyTorch não foi possível nesta rodada porque o ambiente de execução não possui PyTorch; por isso, a análise numérica canônica usa explicitamente os dois JSONs exportados e registra seus hashes.

Tabela 1 – Ambiente e protocolo registrados no manifesto da rodada canônica.

Item	Registro
Sistema operacional	Windows 10 (build 10.0.26200)
Python	3.11.15 (64 bits)
Biblioteca numérica	NumPy 2.4.3; gerador <code>RandomState(42)</code>
Verificador	ESBMC 6.8.0, executável Windows 64 bits
Solver solicitado	Boolector; indisponível no executável Windows
Solver alternativo	Z3 v4.8.9, quando explicitamente indicado
Flags	<code>-no-unwinding-assertions; -boolector</code> ou <code>-z3</code>
Sementes e amostras	semente 42; 10.000 estados para a quantização
Rastreabilidade	<code><evidencias/final_2026/MANIFESTO.json></code>

O executável sem extensão presente no diretório do QNNVerifier é um binário ELF Linux, enquanto `esbmc.exe` é o binário Windows usado pela rodada. O `esbmc.exe` aceita a opção `-boolector`, mas informa em tempo de execução que o solver não foi compilado. Consequentemente, a ausência de Boolector é registrada como bloqueio de infraestrutura, e não como falha de uma propriedade.

4.3 Modelo físico e controlador neural

O estado do CartPole é $s = (x, \dot{x}, \theta, \dot{\theta})$, com posição do carro em metros, velocidade linear em metros por segundo, ângulo do pêndulo em radianos e velocidade angular em radianos por segundo. O atuador recebe uma força contínua $F \in [-10, 10]$ N. A planta usada no simulador emprega os parâmetros $g = 9,8 \text{ m/s}^2$, $m_c = 1,0 \text{ kg}$, $m_p = 0,1 \text{ kg}$, meio comprimento $l = 0,5 \text{ m}$ e passo de Euler $\Delta t = 0,02 \text{ s}$. A dinâmica completa usa seno e cosseno; o harness da propriedade de segurança usa sua linearização $\sin(\theta) \approx \theta$ e $\cos(\theta) \approx 1$.

O ator DDPG tem arquitetura $4 \rightarrow 24 \rightarrow 24 \rightarrow 1$, com ReLU nas duas camadas ocultas e tanh na saída. O vetor de saída é convertido em força por $F = 10 \tanh(z)$. A rede possui 769 parâmetros treináveis. A verificação trata o controlador exportado como o artefato sob análise; ela não constitui prova sobre qualquer outro checkpoint ou sobre o treinamento em geral.

4.4 Representação Q8.8

Para tornar o grafo neural compatível com a aritmética inteira do BMC, foi usada a representação de ponto fixo Q8.8, com escala $S = 256$. Cada valor real é convertido por $q(v) = \text{round}(256v)$. O produto de dois valores escalados é reduzido por divisão inteira por 256. Tanto o código C do harness quanto os scripts de auditoria usam truncamento em direção a zero, de modo a reproduzir o operador $/$ de C.

O domínio discreto empregado pelos harnesses é:

Tabela 2 – Domínio de entrada declarado para os harnesses Q8.8.

Variável	Unidade	Domínio real de referência	Domínio inteiro usado
x	m	$[-2,4, 2,4]$	$[-614, 614]$
$\dot{x}$	m/s	$[-5, 5]$	$[-1280, 1280]$
θ	rad	aproximadamente $[-12^\circ, 12^\circ]$	$[-53, 53]$
$\dot{\theta}$	rad/s	$[-5, 5]$	$[-1280, 1280]$

O limite inteiro 53 representa aproximadamente $11,86^\circ$. Como a quantização por arredondamento pode transformar valores reais próximos de 12° em 54 unidades, o domínio inteiro usado é uma aproximação conservadora e ligeiramente menor que a imagem completa da quantização do intervalo angular. Essa diferença é registrada como ameaça à validade.

A tanh do runtime é aproximada por cinco segmentos lineares, com quebras em $|z| = 64, 192, 384, 768$ unidades Q8.8 e saturação em ± 255 . A mesma função é copiada nos harnesses e na implementação web. Isso elimina uma diferença de implementação entre esses dois artefatos, mas não elimina o erro em relação à tanh real nem à rede Float32.

4.5 Propriedades formais

Foram consideradas as seguintes propriedades do controlador:

1. **Limite do atuador (C).** Para todo estado do domínio, a força quantizada deve satisfazer $-2560 \leq F_Q \leq 2560$, equivalente a $|F| \leq 10 \text{ N}$.

2. **Segurança em um passo (B).** Para todo estado inicial no domínio, calcula-se a ação da rede e um passo da dinâmica angular linearizada. A asserção é $-53 \leq \theta_1 \leq 53$. O horizonte é exatamente um passo; portanto, o resultado não implica invariância para múltiplos passos.
3. **Correção direcional (A).** Para $\theta > 25$ e $\dot{\theta} \geq 0$, testa-se $z > 0$; simetricamente, para $\theta < -25$ e $\dot{\theta} \leq 0$, testa-se $z < 0$. Como a $\tanh$ é monotônica, o sinal de z determina o sinal da força, mas a propriedade continua sujeita à complexidade simbólica da rede.

Na propriedade B, as equações usadas no domínio escalado são

$$\ddot{\theta}_Q = \text{trunc}\left(\frac{4040\theta_Q - 375F_Q}{256}\right), \quad \theta_1 = \theta_Q + \text{trunc}\left(\frac{5\dot{\theta}_Q}{256}\right).$$

Os fatores 4040, 375 e 5 codificam os parâmetros físicos e o passo temporal na escala empregada pelo harness.

4.6 Execução e reprodução

O protocolo canônico usa scripts novos para quantização, ativação, reproduções fixas e verificações universais curtas. A auditoria estrutural acrescenta dois scripts corrigidos para atividade e saturação dos neurônios, cujos nomes, comandos e hashes estão registrados no manifesto. Essas versões constroem o grafo completo da rede a partir das quatro entradas simbólicas. Esses harnesses não impõem bounds de pré-ativação menores que os intervalos calculados e não injetam ativações independentes entre camadas. Como o executável Windows disponível não contém Boolector, a rodada corrigida usa explicitamente Z3 v4.8.9 e registra a indisponibilidade do solver solicitado. Cada processo preserva `stdout`, `stderr`, comando, tempo, status e eventual estado de contraexemplo.

Cada execução grava o comando no manifesto, os arquivos de entrada por hash, os harnesses gerados e os logs correspondentes. Um contraexemplo só é considerado reproduzido quando a mesma entrada inteira e a mesma aritmética Q8.8 produzem a asserção esperada em um harness independente.

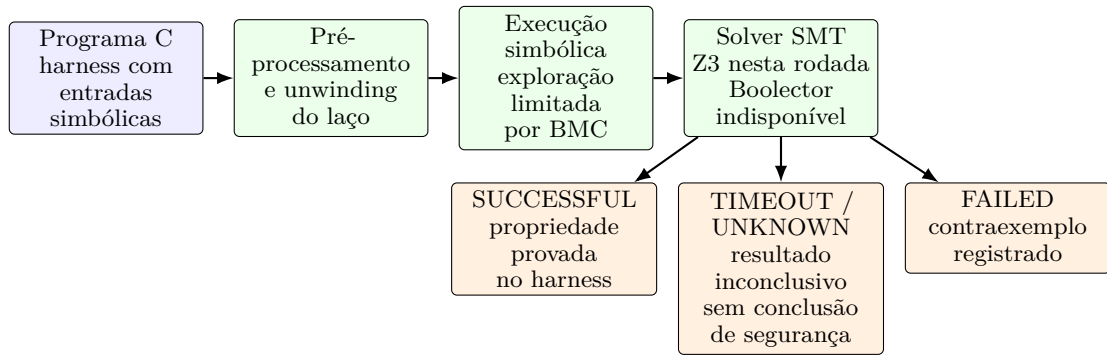


Figura 2 – Fluxo de verificação do ESBMC e interpretação dos veredictos.

A Figura 2 é a chave de leitura dos resultados formais. **SUCCESSFUL** só sustenta a propriedade especificada para o domínio e para o modelo codificado; **FAILED** fornece um contraexemplo; já **TIMEOUT** e **UNKNOWN** não podem ser convertidos em aprovação ou reprovação do controlador.

4.7 Critérios de validade

As conclusões formais são restritas ao checkpoint exportado, à representação Q8.8, ao domínio inteiro da Tabela 2, ao modelo da planta especificado e ao horizonte declarado. A comparação Float32–Q8.8 é estatística sobre 10.000 estados uniformes, não uma prova de equivalência. Resultados **TIMEOUT** ou **UNKNOWN** não recebem interpretação de segurança.

As versões históricas das análises de neurônios mortos e saturação permanecem preservadas no diretório **legacy** das evidências. Os resultados corrigidos foram gerados pelos grafos completos e estão nos dois arquivos JSON correspondentes do diretório **outputs**. Ainda assim, **TIMEOUT** e **UNKNOWN** não são convertidos em provas: a rodada corrigida só permite afirmar uma propriedade quando o respectivo harness termina em **SUCCESSFUL**, ou refutá-la quando termina em **FAILED** com contraexemplo registrado.

5 RESULTADOS E DISCUSSÃO

Os resultados desta seção foram separados por nível de evidência. Os números da rodada canônica estão em `<evidencias/final_2026/outputs>` e podem ser rastreados ao manifesto por hashes. Onde a execução universal não foi concluída, o status é apresentado como inconclusivo. Essa distinção é especialmente importante em verificação baseada em BMC, cuja conclusão depende do programa, do domínio e do solver efetivamente executados Sena et al. (2021).

5.1 Visão geral dos experimentos

Tabela 3 – Resumo da rodada canônica e da rodada estrutural corrigida.

Experimento	Pergunta	Status	Evidência
Quantização	Qual a diferença Float32–Q8.8 em 10.000 estados?	Confirmado	<code><quantization_canonical.json></code> e CSV de extremos
Aproximação tanh	O erro da função usada é inferior a 3%?	Não confirmado	Varredura encontrou máximo de 4,5749%
Reprodução A-direita	O estado publicado realmente gera $z < 0$?	Confirmado para estado fixo	Harness C e log ESBMC
Reprodução B	O estado publicado sai da faixa em um passo?	Confirmado para estado fixo	Harness C e log ESBMC
Limite de força C	A força fica em $[-10, 10]$ N para todo o domínio?	Universal confirmado com Z3	Harness simbólico, Z3 v4.8.9
A-direita universal	A direção correta vale em todo o domínio?	Inconclusivo	Z3 atingiu timeout de 12 s
Atividade dos neurônios	Há contraexemplo de ativação para cada neurônio?	L1: 24/24; L2: 17/24; 7 TIME-OUT	<code><corrected_dead_neurons.json></code>
Saturação das camadas	Há contraexemplo de pré-ativação negativa?	L1: 24/24; L2: 22/24; 2 TIME-OUT	<code><corrected_saturation.json></code>
Saída do controlador	Os dois sinais da saída foram refutados universalmente?	Não: FAILED e TIMEOUT	<code><corrected_saturation.json></code>

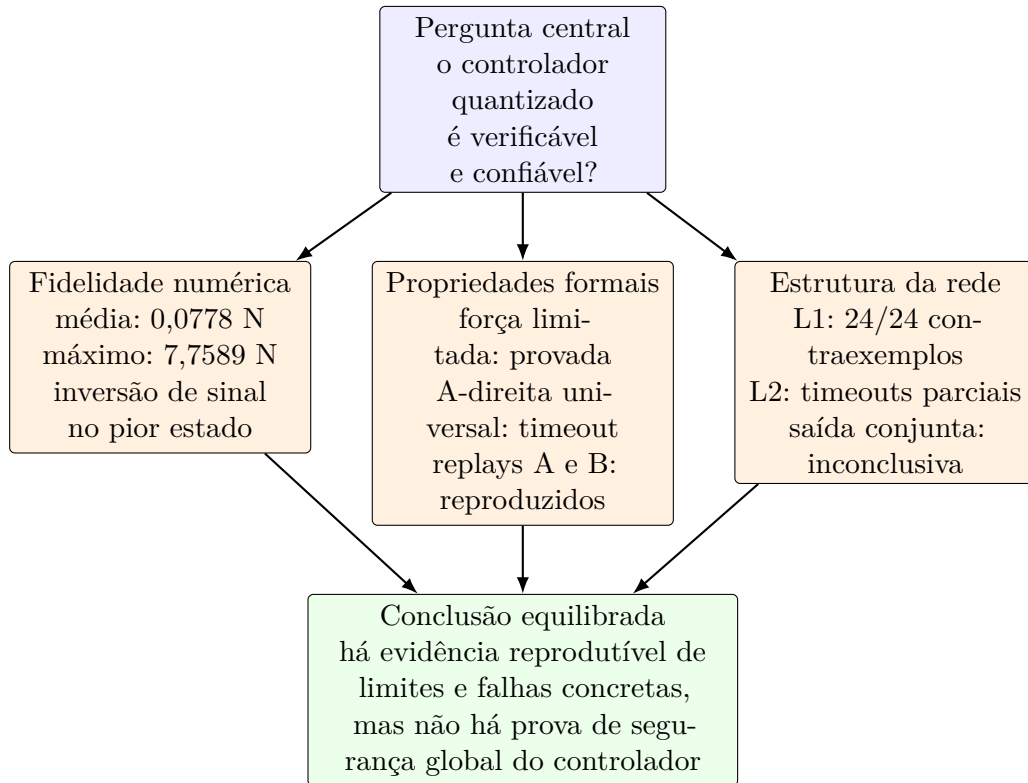


Figura 3 – Síntese visual dos resultados e da conclusão sustentada pelos dados.

A Figura 3 mostra a conexão entre as análises. A quantização apresenta boa média, mas uma cauda de erro capaz de inverter a ação; a verificação formal prova o limite de força, porém não resolve a propriedade direcional universal; e a auditoria estrutural encontra contraexemplos, mas também timeouts. Em conjunto, os resultados justificam uma conclusão de verificabilidade parcial e reprodutível, não uma certificação de segurança global.

5.2 Fidelidade Float32 versus Q8.8

A análise foi reexecutada com `numpy.random.RandomState(42)`, quatro variáveis amostradas uniformemente nos intervalos declarados e 10.000 estados. Os pesos Float32 e Q8.8 foram lidos dos arquivos exportados; seus hashes estão em `<outputs/quantization_canonical.json>`. Os valores coincidem com o relatório histórico dentro do erro de representação decimal.

Tabela 4 – Erro absoluto entre a saída Float32 e a saída Q8.8.

Métrica	Valor
Número de estados	10.000
Escala Q8.8	256
Erro médio	0,0778007759 N
Mediana	0,0390625 N
Percentil 95	0,2262005212 N
Percentil 99	1,0927201962 N
Erro máximo	7,7589197857 N
Máximo relativo à faixa de 10 N	77,5892%

O maior erro ocorreu no estado

$$(-0,8222019265, -2,4912202090, 0,0051761657, 0,7778176171),$$

para o qual a saída Float32 foi +6,4307947857 N e a saída Q8.8 foi -1,328125 N. Portanto, há inversão do sinal da ação nesse caso. O resultado não deve ser descrito como equivalência ou “fidelidade total” ao Float32. A afirmação sustentada é mais restrita: o harness C e o runtime web usam a mesma representação e as mesmas operações Q8.8.

Uma varredura de $z = -10000$ a $z = 10000$, documentada no arquivo JSON da aproximação, encontrou erro absoluto máximo de 0,0457493, isto é, 4,5749% do intervalo unitário da tanh, em $z = -281$. Esse valor contradiz a afirmação anterior de erro inferior a 3%. O erro relativo próximo de zero é mal condicionado e não é utilizado como métrica principal.

5.3 Propriedades formais do controlador

5.3.1 Limite de força

O harness universal da propriedade C usa estados simbólicos no domínio da Tabela 2, a rede inteira Q8.8 e a aproximação de tanh da implementação. Com Z3 v4.8.9, a asserção $-2560 \leq F_Q \leq 2560$ terminou em SUCCESSFUL após 3,5 s. O resultado é uma prova para esse harness e para esse domínio, não uma prova sobre o Float32 nem sobre estados fora dos limites declarados. A propriedade também é esperada pela construção da saída limitada a ± 255 unidades Q8.8, mas a execução simbólica fornece o registro operacional correspondente.

O mesmo teste foi tentado com `-boolector`. O executável Windows disponível respondeu que o solver não foi compilado; esse caso foi registrado como UNKNOWN. Assim, não se deve atribuir a prova desta rodada ao Boolector.

5.3.2 Segurança em um passo e reprodução do contraexemplo

O estado histórico publicado para a propriedade B é

$$s_0 = (-0,7539, -3,9219, -0,1836, -1,5234).$$

Após a quantização, ele corresponde a $(-193, -1004, -47, -390)$. No harness Q8.8, a força calculada é aproximadamente $-9,9609\text{ N}$, e a atualização linearizada resulta em $\theta_1 = -54/256$, aproximadamente $-12,09^\circ$, ultrapassando o limite inteiro -53 . O harness independente `<property_b_safety_replay.c>` teve veredicto **SUCCESSFUL** para a asserção fixa $\theta_1 < -53$, em 0,313 s.

Essa execução confirma a reprodução da violação alegada para o estado fixado. Ela não mostra que todos os estados são inseguros e também não garante que o controlador falhará na dinâmica não linear completa. O que foi verificado é o modelo angular linearizado, em um único passo e na aritmética Q8.8.

5.3.3 Correção direcional e resultados inconclusivos

O estado publicado para A-direita é

$$s_0 = (-0,0117, -4,9219, 0,1094, 0),$$

ou $(-3, -1260, 28, 0)$ em Q8.8. A reprodução independente verificou $z < 0$ para esse estado, contrariando a propriedade que exigia $z > 0$; o harness fixo terminou **SUCCESSFUL** em 0,609 s.

Já o teste universal A-direita, sem bounds artificiais de pré-ativação, atingiu **TIMEOUT** de 12 s com Z3. Isso não é prova nem refutação universal. Há uma divergência documental: o texto de apresentação antigo classifica as duas direções como timeout, enquanto o JSON histórico classifica A-direita e A-esquerda como **FAILED**. A-direita possui estado completo e foi reproduzida; A-esquerda contém apenas $z=27$, sem estado completo, e não é considerada evidência reproduzível.

5.4 Análises estruturais corrigidas

As análises históricas foram refeitas com o grafo completo da rede. As cópias originais estão no diretório `legacy`; os resultados e logs da rodada corrigida estão nos diretórios `outputs` e `logs`. Em particular, a camada 2 é calculada a partir do mesmo vetor simbólico da camada 1, sem injetar ativações independentes, e não há bounds de pré-ativação artificialmente menores que os bounds derivados do grafo.

Tabela 5 – Resultados da rodada estrutural corrigida com Z3.

Propriedade	FAILED	SUCCESSFUL	TIMEOUT	Interpretação
Ativação – camada 1	24	0	0	Cada neurônio tem um contraexemplo ativo
Ativação – camada 2	17	0	7	7 casos permanecem inconclusivos
Não saturação – camada 1	24	0	0	Cada neurônio tem um contraexemplo com $pre < 0$
Não saturação – camada 2	22	0	2	2 casos permanecem inconclusivos
Saída $z \geq 0$	1	0	0	Há contraexemplo para a propriedade universal
Saída $z \leq 0$	0	0	1	A execução terminou por tempo

Na verificação de atividade, **FAILED** significa que o solver encontrou um estado do domínio em que o neurônio está ativo; portanto, os dados sustentam 24/24 contraexemplos na camada 1 e 17/24 na camada 2. Não sustentam, por si sós, que o neurônio esteja ativo em todos os estados. Na camada 2, os sete **TIMEOUT** não podem ser classificados como ativos ou mortos.

Na verificação de saturação, a asserção foi $pre \geq 0$, isto é, saturação positiva em toda a região. Os 24 contraexemplos da camada 1 e os 22 da camada 2 mostram que cada um desses casos possui pelo menos uma entrada com $pre < 0$. Os dois timeouts da camada 2 não autorizam conclusão. Assim, não há neurônio cuja saturação positiva tenha sido provada universalmente nesta rodada.

Para a saída, $z \geq 0$ foi **FAILED** com o estado $(-2, -305, -27, -1184)$ em Q8.8, enquanto $z \leq 0$ atingiu **TIMEOUT** de 4 s. A combinação não permite declarar a saída responsiva em ambos os sinais nem provar uma saturação global; o resultado correto é inconclusivo para a propriedade conjunta.

5.5 Escalabilidade e reprodutibilidade

Os tempos confiáveis desta rodada são os dos harnesses gerados e dos logs correspondentes: 0,609 s e 0,313 s para as reproduções fixas, 3,5 s para a propriedade C universal com Z3 e 12 s até timeout para A-direita universal. Na rodada estrutural corrigida, cada verificação foi limitada a 8 s na análise de atividade e a 4 s na análise de saturação; esses limites estão registrados nos JSONs. Os tempos históricos de 1888, 2722, 90 e 24 não são reutilizados, pois o script atual não os mede nem os grava.

O fluxo demonstra uma diferença relevante entre detectar uma violação concreta e provar uma propriedade para um domínio inteiro. A primeira pode ser reduzida a um harness pequeno e reproduzível; a segunda exige solver disponível, modelo sem restrições

indevidas e tempo suficiente para explorar as combinações das ReLUs. Essa distinção é parte do resultado tecnológico do projeto.

5.6 Discussão integrada, limitações e ameaças à validade

A rodada confirma três contribuições úteis: (i) uma comparação quantitativa e rastreável entre Float32 e Q8.8; (ii) a reprodução independente de dois estados que demonstram comportamentos alegados; e (iii) uma prova simbólica do limite de força com Z3. Ao mesmo tempo, ela identifica limites que precisam aparecer no relatório: a quantização pode inverter a ação em casos extremos, a tanh aproximada tem erro maior que o anunciado, a propriedade de segurança tem horizonte de um passo e o teste direcional não escala na configuração curta. A rodada estrutural corrigida acrescenta contraexemplos de ativação e não saturação para a maior parte dos neurônios, mas mantém sete casos de atividade e dois de saturação como timeouts; portanto, não há base para uma afirmação universal sobre os 48 neurônios.

Há ainda ameaças de modelagem: a dinâmica de B é linearizada, o domínio angular inteiro é ligeiramente menor que a imagem completa do arredondamento de 12 graus, e não foi realizada nesta rodada uma reprodução da trajetória não linear completa. Por fim, o histórico de treinamento contém 1.670 episódios e média final de 100 episódios igual a 472,19, enquanto o gerador web histórico grava 500 e 475,0; esses metadados devem ser reconciliados antes de serem usados como resultado.

6 CONCLUSÃO

O objetivo geral foi avaliar se o ESBMC pode produzir evidências formais e reproduzíveis sobre um controlador neural quantizado, dentro de um domínio de entradas e de um horizonte explicitamente definidos. A rodada canônica confirma que o fluxo pode produzir uma prova de limite de saída para o harness curto e reproduzir estados publicados como contraexemplos, mas não sustenta uma certificação global do controlador. A interpretação permanece limitada pela quantização, pelas hipóteses da planta, pelo horizonte finito, pela correção dos harnesses e pela disponibilidade e custo dos solucionadores SMT. Essa conclusão é compatível com a análise de redes quantizadas por model checking SMT descrita em (SENA et al., 2021; SONG et al., 2022), mas não deve ser interpretada como uma certificação global do controlador Float32 ou da dinâmica não linear completa.

1. **Observação quantitativa de fidelidade.** A rodada canônica, executada com `RandomState(42)`, confirmou em 10.000 estados erro absoluto médio de 0,0778 N, percentil 95 de 0,2262 N, percentil 99 de 1,0927 N e máximo de 7,7589 N. O pior estado registrou inversão do sinal da ação: +6,4308 N em Float32 e -1,3281 N em Q8.8. A varredura da aproximação de tanh registrou erro máximo de 0,0457493 (4,5749% do intervalo unitário). Portanto, Q8.8 não deve ser descrito como equivalente ao Float32 sem qualificar o domínio, a amostragem e a cauda de erro. A origem desses JSONs no checkpoint PyTorch ainda precisa ser confirmada diretamente.
2. **Resultado estrutural histórico, ainda condicional.** Os arquivos legados registram 24 neurônios vivos em cada camada oculta, nenhum neurônio morto e nenhuma saturação permanente na camada analisada. A auditoria, porém, identificou bounds artificiais e independência indevida entre camadas. Assim, esse resultado deve ser tratado como indicação histórica, não como prova universal, até a correção e a repetição do harness. “Vivo” significa apenas que foi encontrada uma entrada do domínio que ativa o neurônio.
3. **Prova formal delimitada.** O harness curto canônico para o limite de força, $|F| \leq 10$ N, terminou **SUCCESSFUL** com Z3 em 3,5 s. Essa é uma prova do programa quantizado e do domínio codificado nesse harness, não do controlador Float32, da dinâmica não linear ou de qualquer implementação que não seja semanticamente equivalente. A tentativa com Boolector terminou **UNKNOWN** porque o solver não está compilado no executável Windows.
4. **Reprodução de contraexemplo, sem refutação universal nova.** A rodada canônica verificou como **SUCCESSFUL** as asserções para os estados fixos publicados

para direção e segurança em um passo. Isso confirma a consistência da asserção de replay com os harnesses gerados, mas não prova por si só que a propriedade universal é falsa. A refutação universal histórica e seus status devem ser publicados somente após preservar um novo log universal e o contraexemplo completo.

5. **Resultado inconclusivo para a propriedade direcional.** Na rodada canônica, a tentativa universal com Z3 atingiu TIMEOUT em 12 s e a tentativa com Boolector terminou UNKNOWN por indisponibilidade do solver. Os registros legados marcam status diferentes; até que o comando, o binário, o solver, o limite de tempo e o log sejam reconciliados, a propriedade deve ser descrita como *inconclusiva*. Timeout ou solver indisponível não são prova de satisfação nem de violação.
6. **Contribuição tecnológica.** O trabalho organiza um fluxo que transforma pesos do controlador em representação Q8.8, gera harnesses C, especifica propriedades para o ESBMC e permite reproduzir contraexemplos no simulador. O valor do fluxo está na rastreabilidade entre especificação, veredicto, estado concreto e comportamento observado, e não em alegar uma garantia além das hipóteses codificadas.
7. **Limites e continuidade.** Permanecem como limites o erro da quantização, a aproximação de tanh, a dinâmica linearizada quando utilizada, o horizonte de um passo, o domínio finito, a correção dos bounds, a dependência do solver e os timeouts. Os próximos passos realistas são validar o checkpoint PyTorch, corrigir os harnesses estruturais, preservar uma execução universal completa, particionar o domínio, avaliar horizontes maiores e comparar a dinâmica linearizada com a não linear. Qualquer decisão sobre patente ou registro de software deve ser tomada apenas após verificar a situação administrativa e a novidade do artefato com a orientação responsável.

Os resultados acima respondem aos objetivos específicos sem extrapolar o domínio formal. Antes da submissão, deve-se substituir as marcações condicionais pela rodada experimental autoritativa, preservar os logs correspondentes e conferir que cada número publicado possui um arquivo bruto, comando e versão associados.

AGRADECIMENTOS

Agradeço à Universidade Federal do Amazonas (UFAM), à Faculdade de Tecnologia e ao grupo de pesquisa pelo apoio institucional, financeiro e técnico ao desenvolvimento deste projeto, bem como ao orientador pela condução do trabalho.

REFERÊNCIAS

BARTO, A. G.; SUTTON, R. S.; ANDERSON, C. W. Neuronlike adaptive elements that can solve difficult learning control problems. *IEEE Transactions on Systems, Man, and Cybernetics*, v. 13, n. 5, p. 834–846, 1983. 13

BIERE, A. et al. Symbolic model checking without bdds. In: *Proceedings of the International Conference on Tools and Algorithms for the Construction and Analysis of Systems*. [S.l.]: Springer, 1999. (Lecture Notes in Computer Science, v. 1579), p. 193–207. 12

CORDEIRO, L. C.; FISCHER, B.; MARQUES-SILVA, J. Smt-based bounded model checking for embedded ansi-c software. *IEEE Transactions on Software Engineering*, v. 38, n. 4, p. 957–974, 2012. 12

GADELHA, M. Y. R. et al. Esbmc 5.0: An industrial-strength c model checker. In: *Proceedings of the 33rd ACM/IEEE International Conference on Automated Software Engineering*. [S.l.]: ACM, 2018. p. 888–891. 12

HUANG, X. et al. Safety verification of deep neural networks. In: *International Conference on Computer Aided Verification*. [S.l.]: Springer, 2017. p. 3–29. 10, 12

LILLICRAP, T. P. et al. Continuous control with deep reinforcement learning. In: *International Conference on Learning Representations*. [s.n.], 2016. Disponível em: <<https://arxiv.org/abs/1509.02971>>. 13

SENA, L. et al. *Verifying Quantized Neural Networks using SMT-Based Model Checking*. 2021. ArXiv preprint arXiv:2106.05997. Disponível em: <<https://arxiv.org/abs/2106.05997>>. 10, 12, 21, 27

SONG, X. et al. Qnnverifier: A tool for verifying neural networks using smt-based model checking. In: *Proceedings of the 44th International Conference on Software Engineering: Companion Proceedings (ICSE '22 Companion)*. [s.n.], 2022. Demo track. Preprint: arXiv:2111.13110. Disponível em: <<https://arxiv.org/abs/2111.13110>>. 10, 13, 27

A CRONOGRAMA EXECUTADO

O quadro abaixo consolida as atividades que possuem registro técnico no repositório. Os períodos iniciais sem documentação detalhada devem ser confrontados com o diário de atividades, relatórios parciais ou registros do orientador antes da submissão.

Tabela 6 – Cronograma de atividades documentadas no projeto.

Período		Atividade registrada ou pendência de comprovação
Setembro–dezembro 2025	de	Vigência inicial do projeto; não foi localizado no repositório um diário mensal consolidado.
Janeiro–abril 2026	de	Continuidade da vigência; completar a partir dos registros do orientador e do bolsista.
Maio de 2026		Evolução da aplicação CartPole, visualização de contraexemplos e documentação da metodologia ESBMC.
Junho de 2026		Implementação do controlador DDPG contínuo, pipeline Q8.8, harnesses e aplicação web interativa.
Julho–agosto 2026	de	Auditoria canônica, correção dos harnesses estruturais, consolidação das evidências e redação do relatório final.